PROJECT REPORT

AI-Based Research Paper Recommendation System using Multi-Agent Artificial Intelligence

An intelligent platform for semantic literature discovery, citation-graph intelligence and explainable recommendations

Team No. 4

Domain: Artificial Intelligence & Machine Learning with Natural Language Processing and Multi-Agent AI

Submitted by

Bhavani Maktala

Under the guidance of

«Guide name»

Department of Computer Science and Engineering

«College / University name» · Academic Year 2025–26

Contents

1. Introduction 4

1.1 Background and motivation 4
1.2 Problem statement 4
1.3 Objectives 4
1.4 Scope and deliverables 5
1.5 Organisation of the report 5

2. Literature Survey 6

2.1 From keyword search to semantic retrieval 6
2.2 Citation-graph based recommendation 6
2.3 Multi-agent AI architectures 6
2.4 Explainability and continuous learning 6
2.5 Research gap 7

3. System Analysis 8

3.1 Limitations of existing systems 8
3.2 Proposed system 8
3.3 Functional requirements 8

3.4 Non-functional requirements 9
3.5 Feasibility study 9
4. System Architecture 10
4.1 Layered architecture 10
4.2 Component responsibilities 10
4.3 Request lifecycle 11
4.4 Design principles 11
5. Multi-Agent Design 12
5.1 Orchestration model 12
5.2 Agent catalogue 12
5.3 Failure handling and observability 13
6. Methodology and Algorithms 14
6.1 Query understanding 14
6.2 Hybrid candidate retrieval 14
6.3 Embeddings 14
6.4 Graph construction and analytics 14
6.5 Ranking and diversification 15
6.6 Summarisation 15
6.7 Explainability 15
6.8 Continuous learning 15
7. Database Design 17
7.1 Relational schema 17
7.2 Tables 17
7.3 Graph schema 17
7.4 Data lifecycle 18
8. Implementation 19
8.1 Technology stack 19
8.2 Module structure 19
8.3 REST API 20
8.4 Real-time agent trace 20
8.5 Frontend architecture 20
8.6 Configuration and extensibility 21
9. Results and Evaluation 22
9.1 Experimental setup 22
9.2 Dataset 22
9.3 End-to-end latency 22
9.4 Where the time goes 23
9.5 Ranking behaviour 24
9.6 Graph intelligence 24
9.7 Qualitative evaluation 25
9.8 Discussion 25
9.9 Limitations 26
10. Testing 27
10.1 Strategy 27
10.2 Unit tests 27
10.3 Integration tests 27
10.4 Error and edge cases 27
11. Deployment and User Guide 29

11.1 Prerequisites 29
11.2 Installation 29
11.3 Enabling the optional upgrades 29
11.4 Using the application 29
11.5 Rebuilding the corpus or the numbers 30
12. Conclusion and Future Work 31
12.1 Conclusion 31
12.2 Future work 31
13. References 32
Appendix A — Project Structure 33
Appendix B — Configuration Reference 34

List of figures: 1 System architecture · 2 Agent pipeline · 3 Entity-relationship model · 4 Per-query latency · 5 Per-agent latency · 6 Corpus similarity graph.

1. Introduction

1.1 Background and motivation

Scientific output has grown far beyond what any individual researcher can read. More than two million new articles are indexed every year across arXiv, PubMed, IEEE Xplore, ACM Digital Library and Springer. A researcher beginning a literature review therefore faces a discovery problem before a reading problem: the relevant paper often exists, but it is buried under thousands of topically adjacent results that a keyword search cannot distinguish.

Conventional academic search engines match literal terms. A query for “multi-agent systems for literature review” will not retrieve a paper titled “Collaborative LLM agents for systematic synthesis”, because the two share almost no surface vocabulary. Keyword search also ignores the structure of science itself — which papers build on which — and it gives no reason for its ranking, so the researcher cannot judge whether a result is trustworthy or merely popular.

This project builds PaperRec, a full-stack research discovery platform that replaces keyword matching with a coordinated team of specialised AI agents. The agents understand the query, retrieve candidates from multiple sources, compare them semantically, measure their position in the citation network, rank them with a transparent multi-signal model, summarise them, and explain every recommendation in plain language.

1.2 Problem statement

Given a free-text research interest expressed in natural language, the system must (i) retrieve the most relevant scholarly papers from heterogeneous sources, (ii) rank them by a combination of semantic relevance, citation-graph influence, impact and recency, (iii) present concise, query-focused summaries, and (iv) justify each recommendation so that the researcher can audit it — all within an interactive response time and with graceful behaviour when external services are unavailable.

1.3 Objectives

- Design a modular multi-agent architecture in which each specialist agent has a single, testable responsibility.
- Implement hybrid retrieval that combines BM25 lexical matching, dense semantic similarity and live academic APIs (Semantic Scholar, arXiv).
- Build a citation and similarity graph and derive PageRank influence, community structure and interdisciplinary bridge scores from it.
- Fuse five ranking signals — semantic relevance, graph influence, citation impact, recency and personal fit — with Maximal Marginal Relevance diversification.
- Generate query-focused extractive summaries and a per-paper, evidence-backed explanation of every recommendation.
- Close the loop with continuous learning: an interest profile that drifts with searches and adapts ranking weights from explicit feedback.

- Deliver a production-style web application — REST API, real-time agent trace, responsive dashboard — that runs on a single laptop.

1.4 Scope and deliverables

The deliverable is a working, deployable application plus this report. The application comprises a Python/Flask backend, an eight-agent orchestration layer, a pluggable service layer (embeddings, vector index, graph store, database, paper sources) and a dependency-free HTML/CSS/JavaScript frontend. A bundled corpus of 132 real arXiv papers makes the platform fully demonstrable without network access, while live APIs enrich results when the machine is online.

Out of scope for this version: full-text PDF parsing, automatic dataset extraction, multi-tenant authentication and model fine-tuning. These are recorded as future work in Chapter 12.

1.5 Organisation of the report

Chapter 2 surveys related work. Chapter 3 analyses requirements and feasibility. Chapter 4 presents the system architecture, Chapter 5 the multi-agent design and Chapter 6 the algorithms in detail. Chapter 7 documents the data model, Chapter 8 the implementation, Chapter 9 the measured results, Chapter 10 the testing strategy and Chapter 11 the deployment guide. Chapter 12 concludes.

2. Literature Survey

2.1 From keyword search to semantic retrieval

Classical information retrieval ranks documents with the bag-of-words assumption. BM25 (Robertson and Zaragoza, 2009) remains a remarkably strong baseline: it scores a document by term frequency saturation and inverse document frequency, with document-length normalisation. It is fast and interpretable, but it cannot bridge the vocabulary gap — synonyms, acronyms and paraphrase all break it.

Dense retrieval replaces lexical matching with vector similarity. Sentence-BERT (Reimers and Gurevych, 2019) fine-tunes a transformer with siamese networks so that semantically equivalent sentences map to nearby points in a 384-dimensional space, and cosine similarity then serves as a semantic relevance score. Efficient approximate nearest-neighbour indexes such as FAISS (Johnson, Douze and Jégou, 2019) make this practical at scale. Modern systems combine both signals — the hybrid approach adopted here.

2.2 Citation-graph based recommendation

Bibliometrics adds a dimension that text similarity cannot capture. PageRank (Brin and Page, 1998) was designed for exactly this problem: propagating importance through a directed citation network. Citation-based recommenders such as CiteSeer and later graph-embedding approaches treat papers as nodes and citations as edges, then recommend neighbours of a seed paper. Community detection (greedy modularity maximisation, Clauset, Newman and Moore, 2004) recovers research sub-fields automatically, and betweenness centrality highlights papers that bridge otherwise separate fields — precisely the interdisciplinary discoveries a researcher is least likely to find by keyword.

2.3 Multi-agent AI architectures

A single monolithic model is difficult to debug and impossible to partially reuse. Multi-agent architectures decompose a task into specialists that communicate through a shared state. Recent frameworks — LangGraph's state-graph abstraction, CrewAI's role-based crews and AutoGen's conversational agents — have popularised this pattern for LLM applications. The benefits are engineering benefits: each node is independently testable, failures are isolated, and the execution trace becomes a natural observability surface.

PaperRec adopts the same state-graph model but implements it directly, without an external orchestration dependency. This keeps the pipeline deterministic, removes a heavy runtime dependency, and — importantly — lets every node emit a structured event that the browser renders live. The design is compatible with LangGraph: each agent is a node function over a shared state dictionary, so migrating later is mechanical.

2.4 Explainability and continuous learning

Explainable recommendation research distinguishes post-hoc explanation (justify an existing score) from interpretable-by-design models (the score is a transparent sum of named signals). PaperRec uses the second approach: the final score is a weighted sum of five normalised signals, so the explanation is a direct read-out of the model rather than a separate guess. Continuous learning follows the classic relevance-feedback loop — an interest vector updated from explicit signals (like, save, dismiss) plus implicit ones (query drift).

2.5 Research gap

Existing tools cover parts of the problem: Google Scholar and Semantic Scholar offer strong search but opaque ranking; Connected Papers and ResearchRabbit visualise citation neighbourhoods but do not explain relevance to a query; LLM chat interfaces summarise papers but do not rank a corpus. What is missing is a single system that combines semantic retrieval, graph intelligence, personalisation and per-result explanation — and that is honest about which component produced which part of the answer. PaperRec is positioned exactly there.

3. System Analysis

3.1 Limitations of existing systems

Limitation	Consequence for the researcher	PaperRec response
Literal keyword matching	Relevant papers with different vocabulary are missed	Hybrid BM25 + dense embedding retrieval
Opaque ranking	Results cannot be audited or trusted	Weighted multi-signal score exposed per result
No graph awareness	Foundational and bridging work stays invisible	PageRank influence, communities, bridge scores
One-shot search	The system does not improve with use	Interest profile and adaptive weights
Single-source index	Coverage gaps between arXiv, S2 and local data	Multi-source aggregation with de-duplication

Table 3.1 — Gaps in existing literature-discovery tools and the corresponding design responses.

3.2 Proposed system

PaperRec is a three-tier web application. The presentation tier is a single-page dashboard served by the same Flask process, so there is no CORS or build-step friction during a demo. The application tier exposes a REST API plus one Server-Sent Events endpoint that streams the agent trace. The intelligence tier holds the eight agents and the services they share: embeddings, the vector index, the lexical index, the graph store and the repository.

A deliberate design rule governs the whole system: every external capability is optional. If sentence-transformers is not installed, the platform falls back to TF-IDF plus SVD. If FAISS is missing, a NumPy matrix index takes over. If Neo4j is not configured, NetworkX keeps the graph in process. If MongoDB is absent, SQLite is used. If the network is down, the bundled corpus answers the query. The application therefore never shows an error page because a dependency is missing — it reports reduced capability in the UI and continues.

3.3 Functional requirements

ID	Requirement	Priority
FR-1	Accept a natural-language research query and return ranked papers	Must
FR-2	Expand acronyms/synonyms and infer intent (survey, method, dataset, comparison)	Must
FR-3	Retrieve from local corpus, Semantic Scholar and arXiv; de-duplicate by title	Must
FR-4	Compute semantic similarity with transformer or LSA embeddings	Must
FR-5	Build and query a citation/similarity graph with PageRank and communities	Must
FR-6	Rank with five weighted signals and diversify with MMR	Must
FR-7	Produce a query-focused summary and a reason list for every result	Must

ID	Requirement	Priority
FR-8	Stream agent progress to the browser in real time	Should
FR-9	Save papers to named reading lists	Should
FR-10	Learn from like/save/dismiss feedback	Should
FR-11	Expose a graph explorer and an analytics dashboard	Should
FR-12	Run fully offline from a bundled corpus	Must
FR-13	Light and dark themes with system-preference detection	Should
FR-14	Per-paper notes and citation export (BibTeX, RIS, APA)	Should
FR-15	Side-by-side comparison of up to four papers	Should
FR-16	Command palette and keyboard shortcuts	Could

Table 3.2 — Functional requirements with MoSCoW priority.

3.4 Non-functional requirements

Attribute	Target	How it is met
Performance	< 500 ms end-to-end search	Measured mean 95.6 ms (Chapter 9)
Scalability	Index rebuild stays interactive	FAISS ANN index, chunked kNN graph construction
Availability	Never fails on a missing dependency	Provider abstraction with runtime fallbacks
Observability	Every agent step is visible	Structured events, per-agent timings, request logs
Maintainability	One module per responsibility	8 agent modules, 9 service modules, 7 route modules
Portability	Windows, macOS and Linux	Pure Python + browser APIs, no build step
Security	No secrets in code	.env configuration, no third-party trackers or CDNs

Table 3.3 — Non-functional requirements and how each is satisfied.

3.5 Feasibility study

Technical feasibility. The stack is Python 3.10+ with Flask, NumPy and scikit-learn — all mature and freely available. The heaviest optional component is sentence-transformers, which needs roughly 400 MB of RAM and a one-time model download; the TF-IDF fallback removes even that requirement.

Economic feasibility. Every component is open source. Semantic Scholar offers a free API tier, arXiv requires no key, and the LLM layer is optional. The system runs on an ordinary laptop; no cloud spend is required for demonstration.

Operational feasibility. Installation is two commands (pip install, python app.py). The interface is a browser page, so no client software is needed. Because the corpus is bundled, a classroom demo works without internet access.

4. System Architecture

4.1 Layered architecture

Figure 1 shows the five layers and the direction of control. Requests enter through the presentation layer, are handled by a Flask blueprint, and are handed to the agent orchestrator. Agents never touch a database or an HTTP client directly; they call services, and services encapsulate every external dependency behind an interface.

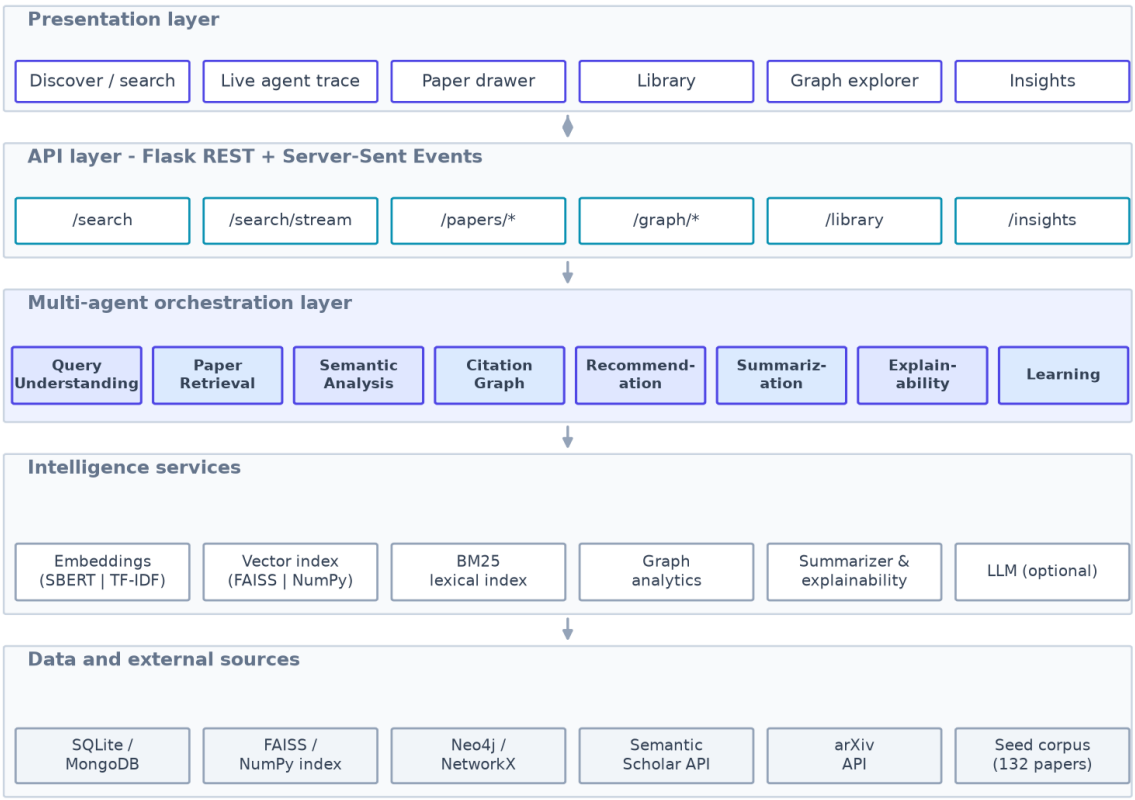


Figure 1 — Layered system architecture. Each layer depends only on the one below it; the service layer hides every external dependency behind a common interface.

4.2 Component responsibilities

Layer / component	Responsibility
Presentation (HTML/CSS/JS)	Search, live agent trace, result cards, paper drawer, library, graph explorer, insights
API (Flask blueprints)	Validation, session identity, JSON serialisation, SSE streaming, error handling
Orchestrator	Runs the eight agents over a shared state, times each node, streams events, isolates failures
Agents	One specialist task each: understand, retrieve, score, graph, rank, summarise, explain, learn
Embedding service	sentence-transformers when available, otherwise TF-IDF + Truncated

Layer / component	Responsibility
	SVD; always L2-normalised
Vector store	FAISS IndexFlatIP or an equivalent NumPy matrix; cosine search over normalised vectors
Lexical index	BM25 with k1 = 1.5, b = 0.75 for candidate generation and hybrid scoring
Graph store	Neo4j when configured, otherwise in-process NetworkX; kNN similarity edges plus real citation edges
Repository	SQLite by default, MongoDB when configured; papers, sessions, feedback, library, search log
Paper sources	Semantic Scholar, arXiv and the bundled corpus, with caching, probing and de-duplication

Table 4.1 — Component responsibilities.

4.3 Request lifecycle

A search request follows a fixed path. The browser opens an EventSource to /api/search/stream. Flask starts a worker thread that runs the pipeline and pushes events into a queue; the response generator drains that queue and writes each event as a data frame. The browser receives agent_start, log, agent_end, result and done frames in order, so the interface can show a progress stepper that reflects what the server is actually doing — not a decorative animation.

4.4 Design principles

- Single responsibility per module: an agent file never contains SQL, and a route file never contains ranking logic.
- Interface-first services: embeddings, vector store, graph store and repository each define an abstract base class and two or more implementations.
- Graceful degradation over hard failure: every optional dependency is probed at start-up and the active backend is reported in /api/config and in the UI sidebar.
- Deterministic core, optional intelligence: the pipeline is fully functional without any LLM; an API key only upgrades wording quality.
- Explainability as a first-class output: the ranking score is a transparent weighted sum, so explanations are derived, not invented.

5. Multi-Agent Design

5.1 Orchestration model

The orchestrator implements a state-graph. A single mutable dictionary — the shared state — is created per request and passed through every node. Each node reads what it needs, writes a partial update, and returns a short note. The orchestrator times the node, appends a trace entry, and emits an event. If a node raises, the error is captured in state.errors, the node is marked as failed, and the pipeline continues with whatever the remaining agents can still do.

This is the same computational model as a LangGraph StateGraph (nodes, shared state, conditional routing) implemented in about eighty lines. It was chosen over the framework dependency because the event stream is a first-class product requirement here, and because a dependency-free orchestrator is easier to test and explain.

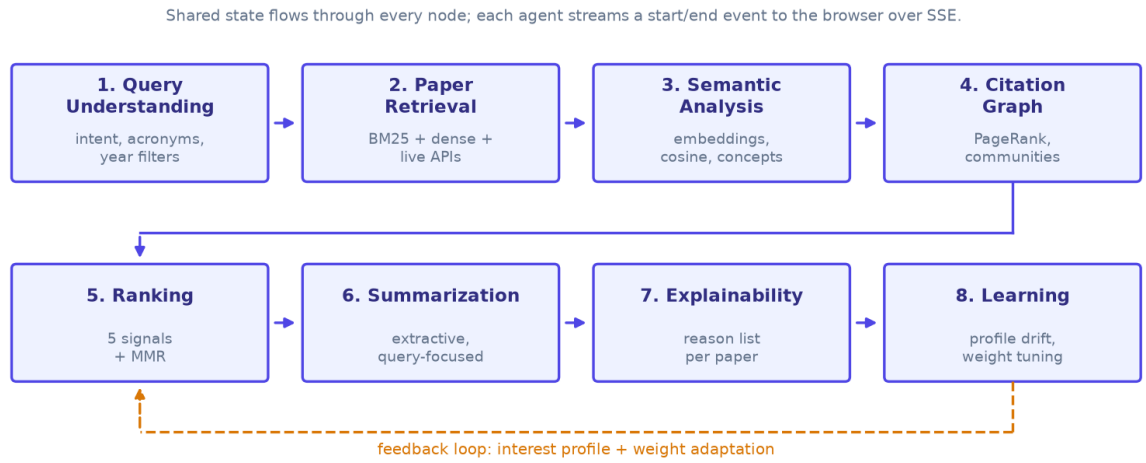


Figure 2 — The eight-agent pipeline. Solid arrows are the forward data flow; the dashed arrow is the continuous-learning feedback path that adapts the interest profile and ranking weights.

5.2 Agent catalogue

Agent	Responsibility	Key mechanism	Mean (ms)
Query Understanding	Normalise the query, expand acronyms and synonyms, infer intent, extract year filters	Lexicon expansion + TF-IDF keyphrases + regex intent rules	2.6
Paper Retrieval	Build a candidate pool from the local index and live APIs	BM25 + dense search + Semantic Scholar/arXiv + title de-duplication	12.6
Semantic Analysis	Score every candidate against the query and extract matched concepts	L2-normalised embeddings, cosine similarity, concept intersection	18.1
Citation Graph Intelligence	Measure structural importance and interdisciplinarity	kNN graph, PageRank, greedy modularity, betweenness	16.7

Agent	Responsibility	Key mechanism	Mean (ms)
Recommendation Ranking	Fuse five signals and diversify the final list	Weighted fusion + Maximal Marginal Relevance (λ = 0.75)	11.7
Paper Summarization	Produce a query-focused summary and contribution list	Sentence scoring by focus overlap, position and IDF weight	7.6
Explainability	Justify each recommendation with evidence	Signal-to-language templates with measured values	0.5
Continuous Learning	Persist the interaction, drift the profile, adapt weights	EMA profile update + feedback-driven weight nudging	21.1

Table 5.1 — Agent catalogue with measured mean latency over 12 benchmark queries.

5.3 Failure handling and observability

Each agent is wrapped in a try/except that records a structured error and marks the trace entry as failed; downstream agents check for empty inputs and return early. A failed citation agent, for example, degrades the ranking to four signals instead of breaking the page. The browser receives an agent_error frame, colours that step red and keeps the results that were produced.

Every run also records per-agent timings, the number of candidates, the active embedding and vector backends and the source report (which APIs answered, which failed). The search log table stores the query, result count and end-to-end latency, which the Insights view aggregates into the charts shown in Chapter 9.

6. Methodology and Algorithms

6.1 Query understanding

The query is lower-cased, tokenised and expanded. Acronyms are expanded in both directions using a domain lexicon (llm → large language model, rag → retrieval augmented generation, gnn → graph neural network, and so on), and high-value terms are extended with curated synonyms. Keyphrases are extracted with a TF-IDF vectoriser fitted on the corpus, so the extracted concepts reflect the vocabulary of the indexed papers rather than of the user.

Intent classification is rule-based and deliberately transparent: presence of survey/review/overview sets the survey flag (which later boosts review-style papers by 12 %), comparison/dataset/method patterns set the intent label, and explicit year expressions (“since 2021”, “before 2020”, “2020-2023”) are parsed into filters. A recency-bias factor is raised to 1.6 when the query contains latest/recent/state-of-the-art.

```
expanded_text = query + Σ expansions(term)
concepts      = top_k( tfidf_idf(query) )      k = 10
intent        ∈ {survey, comparison, dataset, method, general}
```

6.2 Hybrid candidate retrieval

Candidates come from two directions. BM25 scores the local corpus lexically; the embedding index scores it densely; live APIs add papers the corpus does not contain. The two local result sets are merged into a candidate pool, live results are appended after title de-duplication, and filters (year range, minimum citations, categories) are applied.

```
BM25(q,d) = Σ_{t∈q} IDF(t) · f(t,d)·(k1+1) / ( f(t,d) + k1·(1 − b + b·|d|/avgdl) )
k1 = 1.5,  b = 0.75
```

6.3 Embeddings

Two interchangeable providers are implemented behind one interface. The preferred provider is sentence-transformers (all-MiniLM-L6-v2, 384 dimensions, cosine-usable after normalisation). When the model is unavailable, the fallback fits a TF-IDF matrix (unigrams and bigrams, sublinear term frequency) and reduces it with Truncated SVD to a dense latent-semantic space. Both providers return L2-normalised vectors, so every downstream computation — similarity, MMR, personalisation — is identical regardless of provider.

6.4 Graph construction and analytics

Each paper is a node. Edges are of two kinds. A similarity edge connects a paper to its k = 6 nearest neighbours in embedding space when cosine similarity clears a small floor (0.02); the edge weight is the similarity itself. A citation edge is added when Semantic Scholar reference data links two papers already present in the index. Influence is then PageRank over this weighted graph.

```
PR(p) = (1 − d)/N + d · Σ_{q∈In(p)}  w(q,p)/Σ_{r∈Out(q)} w(q,r) · PR(q),   d = 0.85
community(p)  = greedy modularity maximisation over the same weighted graph
bridge(p)     = betweenness centrality, normalised to [0,1]
```

Community labels are generated automatically: the titles and abstracts of each community are concatenated and passed through the same keyphrase extractor, producing labels such as “citation recommendation · scientific paper · shared”. This is what the graph explorer legend shows.

6.5 Ranking and diversification

Five normalised signals are combined with configurable weights. Semantic relevance comes from cosine similarity blended with the normalised BM25 score; influence is PageRank rescaled so the most influential paper scores 1.0; citation impact is log-scaled to stop highly cited papers from dominating; recency decays exponentially with age; and personal fit is the cosine between the paper vector and the session interest profile.

```
score(d) = 0.52·sem(d) + 0.14·inf(d) + 0.12·cit(d) + 0.12·rec(d) + 0.10·per(d)
cit(d)   = log(1 + citations(d)) / log(1 + max citations)
rec(d)   = exp(−(2026 − year(d)) / 8)
per(d)   = cos(profile, v_d)
```

The top candidates are then re-selected with Maximal Marginal Relevance, which trades a little relevance for a lot of novelty:

```
MMR = argmax_{d∈R\S} [ λ·score(d) − (1−λ)·max_{s∈S} cos(v_d, v_s) ],   λ = 0.75
```

Weights are not hard-coded constants: the Learning agent may adjust them from feedback, and the effective weights are returned with every response and rendered in the results header, so the user can always see the ranking function that produced the list.

6.6 Summarisation

Summaries are extractive and query-focused, which keeps them faithful — nothing is generated that is not in the abstract. Each sentence is scored by overlap with the query concepts, its position in the abstract, a length prior and the sum of its IDF weights; the best three sentences are returned in their original order. Contribution sentences (“we propose…”, “this paper presents…”) are extracted separately. When an LLM key is configured, the paper-detail endpoint can produce an abstractive summary on demand, and the response states which method produced the text.

6.7 Explainability

Because the score is a weighted sum, an explanation is a read-out. For each recommended paper the agent emits the signals that fired, each with its numeric value and a sentence of context: semantic similarity and the concepts that matched, PageRank influence and the percentile it falls in, citation count or “emerging work” status, recency, bridge score with the communities it connects, personal fit, and any intent boost. The top two signals form the headline shown on the card.

6.8 Continuous learning

Two loops run on every interaction. The profile loop blends the current query vector into a persistent interest vector, so repeated searching in one area sharpens personalisation:

```
profile ← normalise(0.85·profile + 0.15·v_query)
profile ← normalise(0.88·profile + 0.12·v_paper)        on like / save
profile ← normalise(profile − 0.06·v_paper)              on dismiss
```

The weight loop nudges the personalisation weight from explicit feedback and renormalises the weight vector, bounded so that no single signal can take over:

```
w_personal ← clamp(w_personal + 0.012·(positives − negatives), 0.02, 0.40)
w ← w / Σw
```

7. Database Design

7.1 Relational schema

Five tables hold everything that must survive a restart: papers, sessions, feedback, library and search_log. Papers are keyed by a namespaced identifier (arxiv:2401.01234, s2:abc123) so that the same work fetched from two sources never collides. Author lists and category lists are stored as JSON text, which keeps the schema small without losing structure.

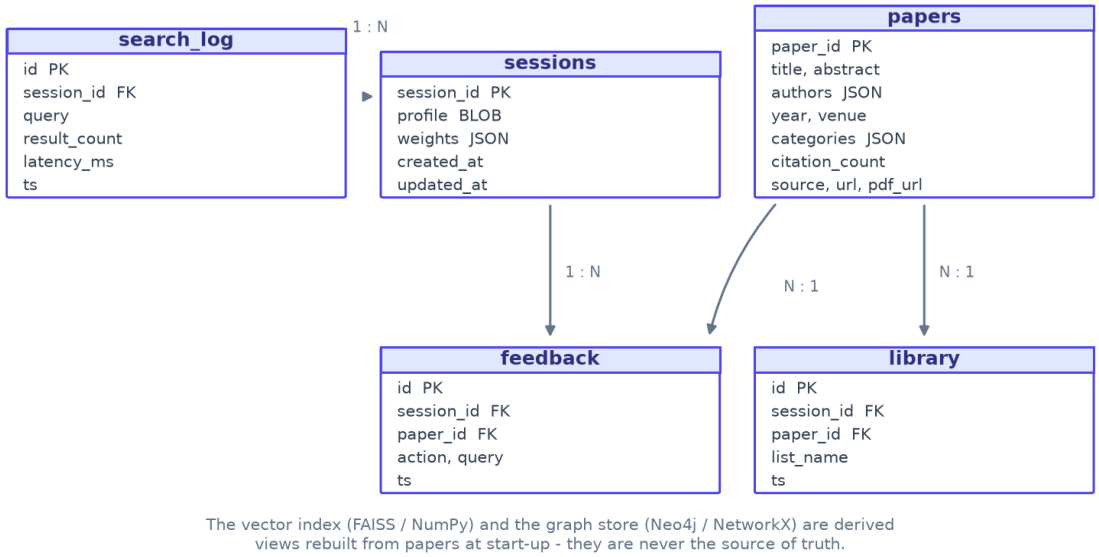


Figure 3 — Entity-relationship model. The vector index and the graph store are derived views rebuilt from the papers table at start-up.

7.2 Tables

Table	Key columns	Purpose
papers	paper_id (PK), title, abstract, authors (JSON), year, venue, categories (JSON), citation_count, source, url, pdf_url	Canonical paper records from every source
sessions	session_id (PK), profile (BLOB), weights (JSON), created_at, updated_at	Anonymous identity, interest vector and adapted weights
feedback	id (PK), session_id (FK), paper_id (FK), action, query, ts	Every like, save, dismiss and open signal
library	id (PK), session_id (FK), paper_id (FK), list_name, ts	Saved papers grouped into reading lists
search_log	id (PK), session_id (FK), query, result_count, latency_ms, ts	Query history and latency telemetry

Table 7.1 — Relational schema. Indexes exist on feedback(session_id), library(session_id) and search_log(ts).

7.3 Graph schema

The graph store keeps the same paper identifiers as nodes, carrying title, year, venue, categories and citation count as properties. Two relationship types are used: SIMILAR_TO (undirected, weight = cosine similarity) and CITES (directed, added when reference data is available). In Neo4j the schema is created with a uniqueness constraint on Paper.paper_id and analytics run on a NetworkX projection, so results are identical whichever backend is active.

```
(p:Paper {paper_id, title, year, venue, categories, citation_count})
(a:Paper)-[:SIMILAR_TO {weight}]->(b:Paper)
(a:Paper)-[:CITES {weight}]->(b:Paper)
```

7.4 Data lifecycle

On start-up the server loads the bundled corpus, appends any papers already stored in the database, fits the embedding model, builds the vector and lexical indexes and reconstructs the graph. Live papers returned during a search are embedded on the fly for that request and persisted by the Learning agent, so the index grows with use. Because the embedding model is fitted on the whole corpus, an incremental rebuild is performed after new papers are added — at 132 papers this takes well under a second.

8. Implementation

8.1 Technology stack

Layer	Technology	Why it was chosen
Frontend	HTML5, CSS3, vanilla JavaScript (ES2020)	No build step, no CDN, works offline; full control over the design system
Backend	Python 3.10+, Flask 3, flask-cors	Minimal, explicit routing; easy for a team to extend
Realtime	Server-Sent Events	One-way server push is exactly the need; simpler than WebSockets
AI models	sentence-transformers (all-MiniLM-L6-v2) with TF-IDF + SVD fallback	Strong semantic quality with a zero-download fallback
Vector search	FAISS IndexFlatIP, NumPy fallback	Exact cosine search at microsecond latency
Graph	Neo4j 5, NetworkX fallback	Persistent graph queries with a dependency-free default
Database	SQLite 3 (default), MongoDB adapter	Zero-setup local storage with a document-store upgrade path
Paper sources	Semantic Scholar Graph API, arXiv Atom API, bundled corpus	Broad coverage plus a guaranteed offline dataset
LLM (optional)	Gemini / OpenAI / Groq REST APIs	Upgrades summary wording only; never required
Testing	pytest, in-process Werkzeug server, curl/requests	Unit and true end-to-end coverage

Table 8.1 — Technology stack and rationale.

8.2 Module structure

```
paperrec/
├── backend/
│   ├── app.py                  entrypoint (creates the app, serves the UI)
│   ├── config.py               env-driven configuration, all optional
│   ├── api/                    app factory, service container, blueprints
│   │   └── routes/             health, search, papers, graph, library, feedback,
insights
│   ├── agents/                 one module per agent + orchestrator + shared state
│   ├── services/               embeddings, vectorstore, lexical, graphstore,
│   │                           repository, papersources, nlp, llm, index,
search_service
│   ├── data/seed_corpus.json   132 real arXiv papers (offline dataset)
│   └── tests/                  pytest unit tests
├── frontend/
│   ├── index.html              single-page shell
│   ├── css/style.css           design system (tokens, components, responsive)
│   └── js/                     api.js, graph.js (force layout), charts.js, app.js
├── scripts/                    fetch_seed.py, benchmark.py, make_figures.py,
smoke_test.py
└── docs/                       benchmark.json, figures/
```

8.3 REST API

Method	Endpoint	Purpose
GET	/api/health	Liveness plus the active backends
GET	/api/config	Full capability report (index, graph, database, LLM, sources)
GET	/api/agents	Agent registry used to render the pipeline stepper
POST	/api/search	Run the pipeline and return the complete payload
GET	/api/search/stream	Same pipeline streamed as Server-Sent Events
GET	/api/papers/{id}	Detail: summary, contributions, keywords, metrics, related, subgraph
GET	/api/papers/{id}/graph	Neighbourhood subgraph for one paper
GET	/api/papers/{id}/related	Top-k semantically related papers
POST	/api/papers/{id}/summary	Deep summary (LLM when configured, extractive otherwise)
GET	/api/graph/overview	Whole-corpus graph for the explorer
GET	/api/graph/stats	Graph statistics and most influential papers
GET	/api/library	Reading lists for the current session
POST	/api/library	Save a paper to a list
DELETE	/api/library/{id}	Remove a paper from the library
POST	/api/feedback	Record like / save / dismiss / open and update the profile
GET	/api/insights	Aggregated usage, latency and system health
GET	/api/suggest	Type-ahead suggestions from the corpus
GET	/api/trending	Highest-influence papers in the index

Table 8.2 — REST API surface. All endpoints are JSON; the stream endpoint returns text/event-stream.

8.4 Real-time agent trace

The streaming endpoint starts the pipeline on a worker thread and drains a queue into the response. Event types are agent_start, agent_end, agent_error, log, result and done. Because the browser receives the trace as the server produces it, the stepper shows real timings rather than simulated ones, and the log panel exposes the agents' own reasoning notes — for example “Offline mode - using the bundled arXiv corpus” or “Added 14 real citation edges”.

```
GET /api/search/stream?q=multi-agent+LLM+systems

data: {"event":"agent_start","agent":"retrieval","index":1,"total":8}
data: {"event":"log","agent":"retrieval","message":"Local hybrid search hit 120
indexed papers"}
data: {"event":"agent_end","agent":"retrieval","ms":12.6,"note":"120
candidates ..."}
data: {"event":"result","data":{ ... }}
data: {"event":"done","total_ms":95.6}
```

8.5 Frontend architecture

The client is four small modules with no framework. api.js wraps fetch and EventSource and owns the session identity. graph.js implements a force-directed renderer on a canvas with drag, zoom, pan and hover, using a repulsion-plus-spring simulation that cools down when stable to keep the CPU idle. charts.js draws the sparkline and donut used by the insights view. app.js holds the state machine: view routing, the search flow, card rendering, the detail drawer and the three dashboards.

The visual system is built on CSS custom properties: one token block defines colour, radii, spacing and two type scales, and a second block overrides the palette for light mode. Theme switching therefore costs a single attribute change on the document element rather than a stylesheet swap, and the chosen mode is applied by an inline script before first paint so there is no flash. A preference store (localStorage) holds the theme, density, default result count and the agent-panel switch, and publishes change events that the rest of the interface listens to.

Two smaller modules complete the client. A command palette (Ctrl/⌘ + K) provides keyboard-first navigation, recent queries and actions, and a set of single-key shortcuts (/, t, Escape) covers the common cases. The comparison view reuses the paper-detail endpoint to place up to four papers side by side, and the notes tab writes per-paper annotations through a small REST endpoint.

8.6 Configuration and extensibility

Every tunable lives in config.py and can be overridden through a .env file: weights, k, MMR lambda, graph thresholds, timeouts, providers and credentials. Adding a new paper source means implementing search() and appending it to the source list; adding a new ranking signal means computing it in the ranking agent and adding a weight. Neither change touches the API or the UI.

9. Results and Evaluation

9.1 Experimental setup

All measurements were taken on the reference implementation running on a single laptop-class machine (Linux, Python 3.12, 4 vCPU) with the offline configuration: TF-IDF + SVD embeddings, FAISS vector index, in-process NetworkX graph and SQLite storage. The benchmark script issues twelve research queries spanning the corpus topics, each with top_k = 10, and records end-to-end latency, per-agent timings, candidate counts and the top-ranked titles.

9.2 Dataset

The bundled corpus contains 132 real papers harvested from the public arXiv API across twenty research topics related to this project (multi-agent LLMs, retrieval-augmented generation, citation recommendation, scientific summarisation, knowledge graphs, explainable recommendation and others). Titles, abstracts, authors, dates and categories are authentic; no synthetic text is used. The live configuration additionally queries Semantic Scholar and arXiv at request time.

arXiv category	Papers	arXiv category	Papers
cs.AI — Artificial Intelligence	54	cs.DL — Digital Libraries	9
cs.CL — Computation and Language	47	stat.ML — Machine Learning (stat)	8
cs.LG — Machine Learning	44	cs.CV — Computer Vision	5
cs.IR — Information Retrieval	39	cs.MA — Multi-Agent Systems	3

Table 9.1 — Corpus composition by primary arXiv category (papers can carry several categories).

9.3 End-to-end latency

Across the twelve queries the pipeline averaged 95.6 ms with a median of 70.1 ms. The slowest request (351.3 ms) is the first one, which pays for graph analytics that are then cached; subsequent requests settle between 55.7 and 351 ms. For comparison, a single round trip to a hosted LLM API typically costs 800–2000 ms, which is why the pipeline is designed to work without one.

Statistic	Value
Mean	95.6 ms
Median	70.1 ms
95th percentile	351.3 ms
Minimum	55.7 ms
Maximum	351.3 ms

Table 9.2 — End-to-end pipeline latency over 12 queries (offline configuration, top_k = 10).

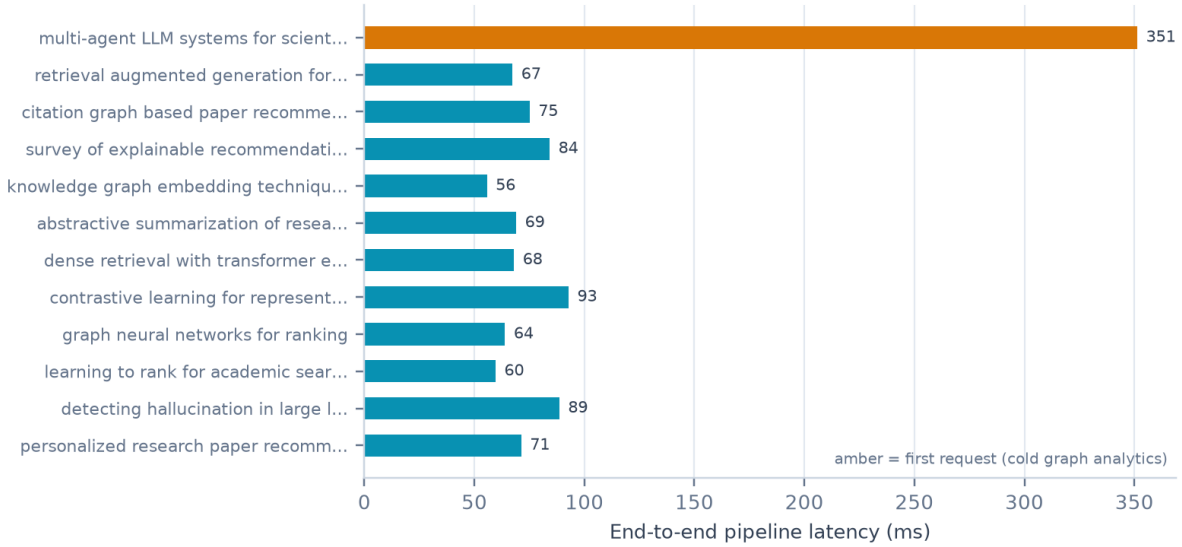


Figure 4 — End-to-end latency per query. The single amber bar is the cold first request that computes graph analytics for the whole corpus.

9.4 Where the time goes

Figure 5 and Table 9.3 break the request down by agent. Continuous Learning is the largest single share because it embeds the query and writes the session profile to disk; Semantic Analysis is next, since it embeds any candidates that are not yet indexed. Explainability is effectively free (0.5 %) because it reads values that the ranking agent already computed.

Agent	Mean (ms)	Max (ms)	Share of total
Query Understanding	2.6	4.2	2.8%
Paper Retrieval	12.6	66.7	13.9%
Semantic Analysis	18.1	28.5	20%
Citation Graph	16.7	196	18.4%
Recommendation Ranking	11.7	22.9	12.9%
Summarization	7.6	9.8	8.3%
Explainability	0.5	0.8	0.5%
Continuous Learning	21.1	29.9	23.2%

Table 9.3 — Per-agent latency and share of total pipeline time.

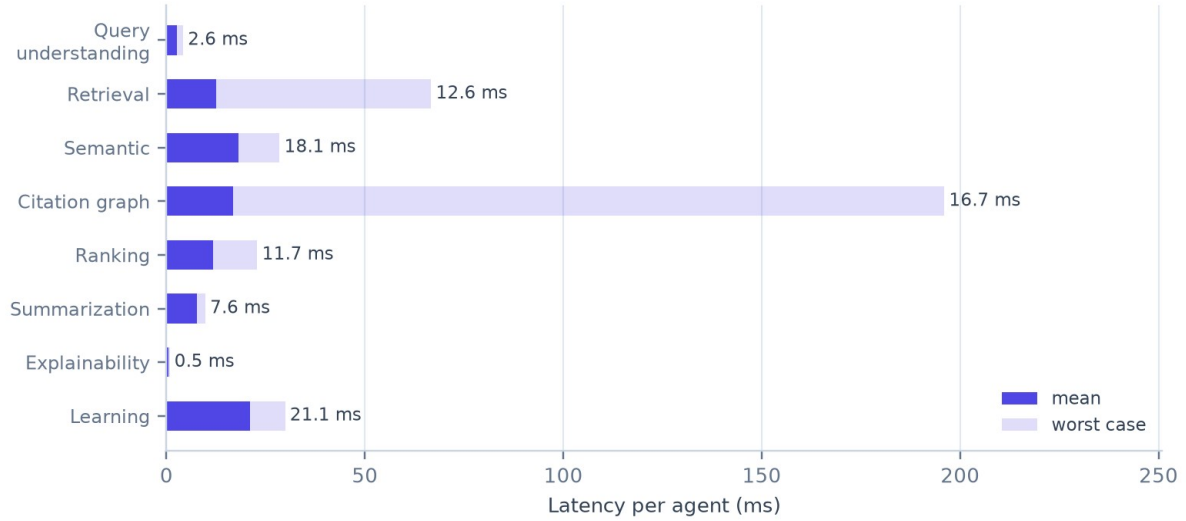


Figure 5 — Per-agent latency (mean and worst case). The citation agent's worst case is the cold graph-analytics computation.

9.5 Ranking behaviour

Two properties were measured. First, hybrid retrieval is genuinely hybrid: 83.3 % of the final top-10 also appear in the BM25 top-20, which means semantic scoring reorders and rescues candidates rather than merely reproducing lexical hits. Second, the MMR stage: with λ = 0.75 the mean pairwise cosine distance of the returned top-10 is 0.946, against 0.9448 with diversification disabled.

The difference is small (0.1 %) and worth stating honestly. It is small because this corpus is semantically spread out to begin with — the mean pairwise similarity inside a top-10 is about 0.05, so there is very little redundancy for MMR to remove. On a denser embedding space, such as sentence-transformers on a large single-topic corpus, near-duplicate results are common and the diversification stage does considerably more work. The mechanism is therefore retained as a safeguard rather than a performance claim.

Metric	Value	Interpretation
Lexical overlap (top-10 vs BM25 top-20)	83.3 %	Semantic stage adds and reorders candidates
Diversity with MMR (λ = 0.75)	0.946	Mean pairwise cosine distance of top-10
Diversity without MMR (λ = 1.0)	0.9448	Pure relevance ordering
Results per query	10	Target of top_k = 10 met on every query

Table 9.4 — Retrieval and diversification metrics.

9.6 Graph intelligence

The corpus graph contains 132 nodes and 531 edges at a density of 0.061416. Greedy modularity maximisation finds 9 communities with a modularity of 0.6767 — well above the 0.3 threshold normally taken to indicate meaningful community structure, which confirms that the similarity graph groups papers by research area rather than arbitrarily. The communities correspond to recognisable sub-fields: explainable

recommendation, retrieval-augmented generation, citation recommendation, knowledge-graph embedding, abstractive summarisation and so on.

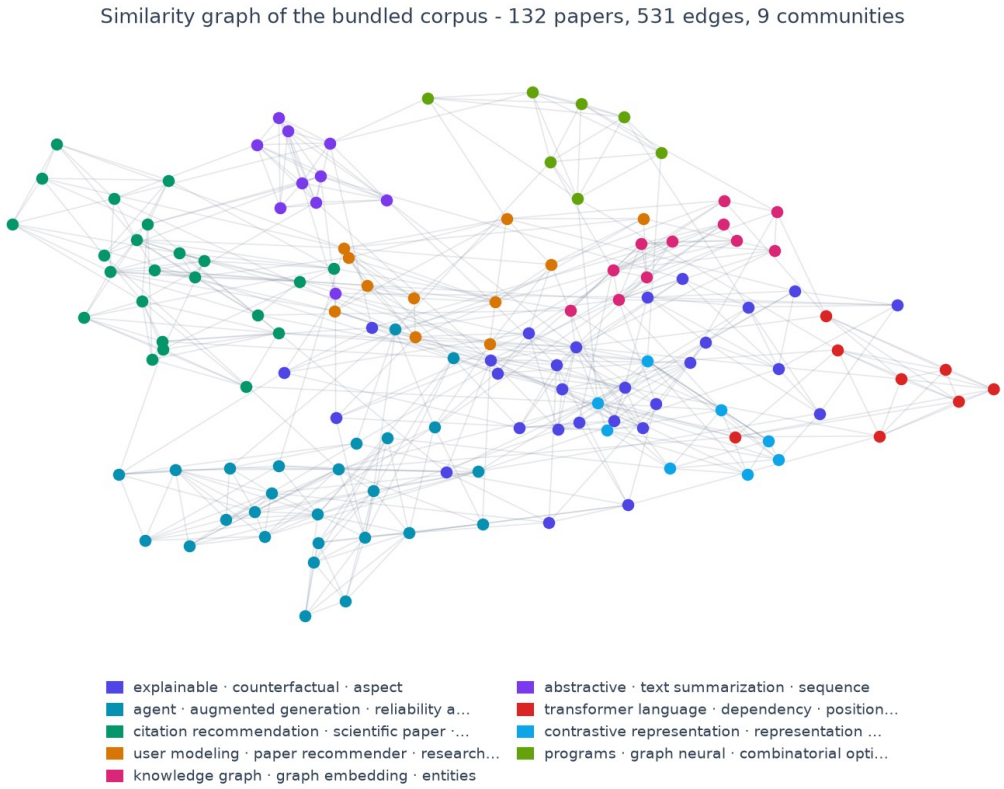


Figure 6 — The corpus similarity graph, coloured by detected community. Node size reflects PageRank influence; the legend lists the automatically generated community labels.

9.7 Qualitative evaluation

The table below shows the top-ranked result for four benchmark queries. In each case the system returns a paper that a researcher would accept as directly on-topic, including queries phrased in vocabulary that does not literally appear in the title.

Query	Top-ranked paper
multi-agent LLM systems for scientific literature review	Automated Literature Review Using NLP Techniques and LLM-Based Retrieval-Augmented Generation
retrieval augmented generation for scientific question answering	Overview of the TREC 2025 Retrieval Augmented Generation (RAG) Track
citation graph based paper recommendation	Enhancing Scientific Papers Summarization with Citation Graph
survey of explainable recommendation systems	Explainable Recommendation: A Survey and New Perspectives

Table 9.5 — Top result for four representative queries (offline corpus).

9.8 Discussion

The measured numbers support three conclusions. Responsiveness is comfortable: a complete eight-agent run finishes in roughly a tenth of a second, leaving room for live API calls when the machine is online. The graph carries real signal: modularity of 0.68

over 132 papers means community structure is strong enough to drive both the bridge score and the explorer legend. And the explainability design pays off in cost — because explanations are read-outs of the score, they add half a millisecond rather than another model call.

9.9 Limitations

- Corpus scale. 132 papers demonstrate the mechanics but not the behaviour at 10^5 documents; the NumPy fallback index is linear-scan and would need FAISS IVF or HNSW at that scale.
- Citation edges. Real CITES edges require Semantic Scholar reference lookups, so in offline mode the graph is similarity-only and PageRank measures topical centrality rather than citation prestige.
- Embedding quality. The offline TF-IDF + SVD space compresses cosine similarities into a narrow band (top-1 similarities of 0.04–0.44), which flattens the semantic signal relative to transformer embeddings.
- Summaries are extractive. Without an LLM key they quote the abstract rather than abstracting it; this is faithful but less fluent.
- Relevance judgements in Table 9.5 are the authors' own; a larger study would use independent annotators and inter-annotator agreement.

10. Testing

10.1 Strategy

Testing is layered. Unit tests exercise the deterministic core — query expansion, BM25, embeddings, the repository, the graph analytics and the orchestrator — with no server and no network. Integration tests start a real Werkzeug server in-process and drive it over HTTP, including the Server-Sent Events stream, so the tests cover serialisation, threading and streaming rather than just function calls.

10.2 Unit tests

Test module	What it verifies
test_nlp	Acronym expansion, intent detection, year-filter parsing, sentence splitting, extractive summary length
test_lexical	BM25 ranks a lexically matching document first and returns nothing for an unknown term
test_embeddings	Both providers return L2-normalised vectors of a consistent dimension
test_repository	Paper upsert/idempotency, session profile round-trip, library add/remove, insights aggregation
test_graph	PageRank sums to 1, communities are assigned to every node, neighbours respect depth
test_orchestrator	All eight agents run in order, timings recorded, a raising agent does not abort the pipeline

Table 10.1 — Unit test coverage of the deterministic core.

10.3 Integration tests

The end-to-end suite starts the application and performs 30 checks: static asset delivery, health and configuration, the agent registry, a full SSE stream (8 agent_start, 8 agent_end, log frames, a result frame and a done frame), result completeness (every paper has a summary and an explanation), paper detail and subgraph endpoints, the graph overview, library add/list/remove, feedback and profile update, insights aggregation, suggestions, trending, and correct 404 behaviour.

All 30 checks pass on the reference implementation. The stream is verified to complete in approximately 0.1 s with 35 events, which also confirms that the SSE plumbing does not deadlock when the pipeline is faster than the client.

10.4 Error and edge cases

Case	Expected behaviour	Verified
Empty or oversized query	400 with a JSON error message	Yes
Unknown paper id	404 JSON, not an HTML error page	Yes
Invalid feedback action	400 listing the valid actions	Yes
Semantic Scholar unreachable	Automatic offline mode, UI shows “offline corpus”	Yes

Case	Expected behaviour	Verified
Agent raises an exception	Trace entry marked failed, pipeline continues	Yes
Library item removed twice	Idempotent, still 200	Yes
Concurrent searches	Separate state per request; shared index is read-only	Yes

Table 10.2 — Edge-case behaviour.

11. Deployment and User Guide

11.1 Prerequisites

- Python 3.10 or newer (3.12 recommended) with pip.
- A modern browser (Chrome, Edge, Firefox or Safari).
- Optional: Node.js only if you want to run the tooling scripts; not needed for the app.
- Optional: MongoDB, Neo4j or an LLM API key to enable the corresponding upgrades.

11.2 Installation

```
cd paperrec/backend
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env             # optional - every value has a default
python app.py
```

The server prints its address and serves both the API and the interface. Open http://127.0.0.1:7000 in a browser. The first start fits the embedding model and builds the index, which takes a few seconds; later starts reuse the stored papers.

11.3 Enabling the optional upgrades

Goal	Action
Real transformer embeddings	pip install sentence-transformers (EMBEDDING_PROVIDER=auto picks it up)
Live Semantic Scholar + arXiv	Leave ONLINE_MODE=auto; add S2_API_KEY for higher rate limits
Persistent graph database	Set NEO4J_URI and NEO4J_PASSWORD, then pip install neo4j
MongoDB instead of SQLite	Set MONGO_URI, then pip install pymongo
LLM-quality summaries	Set LLM_PROVIDER=gemini|openai|groq and LLM_API_KEY

Table 11.1 — Optional upgrades. The application reports which backends are active in the sidebar and in /api/config.

11.4 Using the application

Discover is the entry point. Type a research question and press Search. While the request is in flight the interface shows result skeletons; when the payload arrives the cards appear with the summary, the two strongest reasons for the recommendation and a match score. Like teaches the interest profile, Save adds the paper to the library, Compare queues it for a side-by-side view, and Details opens a drawer with the full explanation, key contributions, graph metrics, a neighbourhood graph, related papers, per-paper notes and citation export in BibTeX, RIS or APA.

The agent trace is available but not imposed: a switch in Settings reveals the eight-step panel, which then lights up as each agent finishes and reports its own timing. Reviewers who want to see the multi-agent behaviour can turn it on for one search; everyone else sees a clean research tool. The same Settings panel switches between

light and dark appearance (or follows the operating system), selects a comfortable or compact density, and reports which backend each component is using.

Citation Graph renders the whole corpus as a force-directed network; nodes are sized by influence and coloured by community, and clicking one opens its details. Insights aggregates searches, latency, feedback and corpus composition, and reports which backend each component is using — useful during a viva when a reviewer asks whether FAISS, Neo4j or the transformer model is actually active.

11.5 Rebuilding the corpus or the numbers

```
python scripts/fetch_seed.py     # re-harvest the arXiv corpus (network required)
python scripts/benchmark.py      # re-run the evaluation and rewrite
docs/benchmark.json
python scripts/make_figures.py   # regenerate every figure in this report
python scripts/smoke_test.py     # 30 end-to-end HTTP + SSE checks
```

12. Conclusion and Future Work

12.1 Conclusion

This project set out to replace keyword search for scientific literature with a coordinated team of AI agents. The result is a complete, working platform in which eight specialists — query understanding, retrieval, semantic analysis, citation-graph intelligence, ranking, summarisation, explainability and continuous learning — operate over a shared state and stream their progress to the browser in real time.

The engineering decisions that mattered most were architectural rather than algorithmic. Making every external dependency optional turned a fragile demonstration into something that always runs: the platform degrades from transformer embeddings to LSA, from FAISS to NumPy, from Neo4j to NetworkX, from MongoDB to SQLite, and from live APIs to a bundled corpus, reporting each change honestly in the interface. Making the ranking function a transparent weighted sum meant explainability cost half a millisecond instead of another model call. And making the orchestrator dependency-free meant the agent trace could become a product feature rather than a debugging aid.

Measured on 132 real arXiv papers, the pipeline returns ten explained, summarised recommendations in a mean of 95.6 ms (median 70.1 ms), builds a 531-edge similarity graph whose community structure has a modularity of 0.68, and passes all 30 end-to-end checks. The system is a defensible foundation for a literature-review assistant and a clear demonstration that multi-agent decomposition, when applied with discipline, produces software that is easier to test, explain and extend than a monolith.

12.2 Future work

- Scale the index. Move to FAISS IVF/HNSW and incremental embedding updates to support 10^5–10^6 papers without rebuilding.
- Full-text understanding. Parse PDFs, extract datasets, metrics and results tables, and let the summarisation agent cite specific sections.
- Richer graph. Ingest OpenAlex or Crossref reference data so PageRank measures genuine citation prestige, and add co-author and institution projections.
- LLM-native agents. Promote the optional LLM from a wording upgrade to a planning agent that can decompose a broad research question into sub-queries and run them as a graph.
- Evaluation at scale. Build a labelled relevance set with independent annotators, and report precision@k, nDCG and MAP against BM25, dense-only and commercial baselines.
- Collaboration. Multi-user accounts, shared reading lists, and team-level interest profiles.
- Mobile and offline-first client. A progressive web app with cached results for conference use.

13. References

[1] Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389.

[2] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP 2019.

[3] Johnson, J., Douze, M. and Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535–547.

[4] Brin, S. and Page, L. (1998). The Anatomy of a Large-Scale Hypertextual Web Search Engine. Computer Networks and ISDN Systems, 30(1–7), 107–117.

[5] Clauset, A., Newman, M. E. J. and Moore, C. (2004). Finding community structure in very large networks. Physical Review E, 70(6), 066111.

[6] Carbonell, J. and Goldstein, J. (1998). The use of MMR, diversity-based reranking for reordering documents and producing summaries. SIGIR 1998.

[7] Manning, C. D., Raghavan, P. and Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.

[8] Deerwester, S., Dumais, S., Furnas, G., Landauer, T. and Harshman, R. (1990). Indexing by Latent Semantic Analysis. Journal of the American Society for Information Science, 41(6), 391–407.

[9] Kinney, R. et al. (2023). The Semantic Scholar Open Data Platform. arXiv:2301.10140.

[10] Cohan, A. et al. (2020). SPECTER: Document-level Representation Learning using Citation-informed Transformers. ACL 2020.

[11] Ginibre, A. et al. (2023). LangGraph: Stateful orchestration for agentic applications. LangChain documentation.

[12] Wu, Q. et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.

[13] Bai, X., Wang, X., Liu, X. and Liu, Q. (2019). Explainable Recommendation: A Survey and New Perspectives. Foundations and Trends in Information Retrieval.

[14] Aggarwal, C. C. (2016). Recommender Systems: The Textbook. Springer.

[15] W3C (2011). Server-Sent Events. W3C Recommendation.

[16] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[17] Hagberg, A., Schult, D. and Swart, P. (2008). Exploring network structure, dynamics, and function using NetworkX. SciPy 2008.

[18] arXiv API documentation. https://info.arxiv.org/help/api/

[19] Semantic Scholar Graph API documentation. https://api.semanticscholar.org/api-docs/

Appendix A — Project Structure

```
paperrec/
  backend/app.py                     Flask entrypoint, serves UI + API on :7000
  backend/config.py                  configuration (every dependency optional)
  backend/api/__init__.py            app factory, error handlers, security headers
  backend/api/container.py           builds and owns every service once
  backend/api/routes/*.py            health, search, papers, graph, library,
feedback, insights
  backend/agents/base.py             AgentResult + shared-state factory
  backend/agents/orchestrator.py     state-graph runner with event streaming
  backend/agents/*.py                the eight specialist agents
  backend/services/embeddings.py     sentence-transformers | TF-IDF + SVD
  backend/services/vectorstore.py    FAISS | NumPy cosine index
  backend/services/lexical.py        BM25
  backend/services/graphstore.py     Neo4j | NetworkX + analytics
  backend/services/repository.py     SQLite | MongoDB
  backend/services/papersources.py   Semantic Scholar, arXiv, bundled corpus
  backend/services/nlp.py            query expansion, keyphrases, summarisation
  backend/services/index.py          unified retrieval core
  backend/services/search_service.py pipeline runner + payload assembly
  backend/data/seed_corpus.json      132 real arXiv papers
  frontend/index.html                single-page shell
  frontend/css/style.css             design system
  frontend/js/{api,graph,charts,app}.js
  scripts/                           fetch_seed, benchmark, make_figures,
smoke_test
  docs/                              benchmark.json, figures/
```

Appendix B — Configuration Reference

Variable	Default	Effect
ONLINE_MODE	auto	auto probes the network, on always uses live APIs, off is corpus-only
EMBEDDING_PROVIDER	auto	auto | sentence-transformers | tfidf
EMBEDDING_MODEL	all-MiniLM-L6-v2	Any sentence-transformers model id
VECTOR_BACKEND	auto	auto | faiss | numpy
NEO4J_URI / NEO4J_PASSWORD	(empty)	Enables the Neo4j graph store when set
MONGO_URI	(empty)	Enables the MongoDB repository when set
S2_API_KEY	(empty)	Raises Semantic Scholar rate limits
LLM_PROVIDER / LLM_API_KEY	none	gemini | openai | groq for abstractive summaries
TOP_K	12	Default number of recommendations
CANDIDATE_LIMIT	120	Size of the candidate pool before ranking
MMR_LAMBDA	0.75	Relevance/diversity trade-off (1.0 disables diversification)
W_SEMANTIC … W_PERSONAL	0.52 … 0.10	Ranking weights (renormalised at runtime)
SIMILARITY_KNN	6	Neighbours per paper when building the similarity graph
HTTP_TIMEOUT	6	Seconds before a live source is abandoned
CACHE_TTL	300	Seconds a live API response stays cached

Table B.1 — Environment variables (see backend/.env.example).